

Lab Assignment-4

Trees & Graphs

Course: Basics of Data Structures | Code: ETCCBD201
School of Engineering & Technology

Learning Outcomes Addressed

This report covers the implementation and conceptual explanation of three fundamental data structures: **Binary Search Tree (BST)**, **Graph with BFS & DFS traversals**. It includes time-complexity analysis and real-world use-case discussion.

Task 1 — Binary Search Tree (BST)

1.1 What is a Binary Search Tree?

A Binary Search Tree is a tree data structure where each node holds a key and satisfies the **BST property**: every key in the **left subtree** is *smaller* than the node's key, and every key in the **right subtree** is *larger*. This ordering property makes searching very efficient — at each node we can immediately decide whether to go left or right, halving the remaining search space on average.

1.2 Why Inorder Traversal Prints Keys in Sorted Order

Inorder traversal visits nodes in the sequence: **Left** → **Root** → **Right**.

Because of the BST property, the left subtree always contains smaller keys and the right subtree always contains larger keys. So by recursively visiting the left subtree first, then the current node, then the right subtree, we always output keys from smallest to largest — producing a perfectly sorted sequence without any extra sorting step.

Example — Insert [50, 30, 70, 20, 40, 60, 80] → Inorder gives: 20, 30, 40, 50, 60, 70, 80

1.3 BST Delete — Three Cases Explained

Case	Condition	Action Taken
Case 1	Leaf node (no children)	Simply remove the node — no restructuring needed.
Case 2	One child (left or right)	Replace the node with its only child directly.
Case 3	Two children	Find the in-order successor (smallest key in right subtree), copy its value, and delete the successor node.

1.4 Time Complexity — BST Operations

The efficiency of BST operations depends on the **height** of the tree.

Operation	Average Case	Worst Case (Skewed Tree)	Explanation
Insert	$O(\log n)$	$O(n)$	Each level roughly halves the candidates (avg); skewed tree is linear.
Search	$O(\log n)$	$O(n)$	Same reasoning as insert.
Delete	$O(\log n)$	$O(n)$	Finding the node is $O(\log n)$; finding in-order successor is at most $O(n)$.
Inorder	$O(n)$	$O(n)$	Must visit every node exactly once.

In simple terms: A balanced BST (height $\approx \log n$) is very fast. If you always insert keys in sorted order, the tree grows like a chain (worst case) and performance drops to $O(n)$. Self-balancing trees like AVL or Red-Black trees fix this automatically.

Task 2 — Graph Representation + BFS & DFS

2.1 Graph and Adjacency List Representation

A **graph** is a collection of **nodes (vertices)** connected by **edges**. In this task the graph is **directed** (edges have direction) and **weighted** (each edge has a numeric cost/distance).

We store the graph as an **adjacency list**: a dictionary where each key is a node and its value is a list of (neighbour, weight) tuples. This is space-efficient — it only stores existing edges, making it ideal for sparse graphs (few edges relative to the number of possible edges).

Node	Outgoing Edges (neighbour, weight)
A	B(2), C(4)
B	D(7), E(3)
C	E(1), F(8)
D	F(5)
E	D(2), F(6)
F	(no outgoing edges)

2.2 BFS vs DFS — Simple Explanation

Aspect	BFS (Breadth-First Search)	DFS (Depth-First Search)
Strategy	Explore all neighbours first, then go deeper.	Go as deep as possible along one path before backtracking.
Data Structure Used	Queue (FIFO — first in, first out)	Stack / Recursion call stack (LIFO)
Result from A	A → B → C → D → E → F	A → B → D → F → E → C
Memory Usage	Can use more memory for wide graphs (stores all nodes on a level).	Uses less memory for wide graphs; more on deep/tall graphs.

2.3 Real-World Use Cases

Algorithm	Real-World Use Case	Why This Algorithm?
BFS	Finding the shortest path in an unweighted graph (e.g., social network).	BFS always explores nodes closest to the start first.
DFS	Cycle detection in a dependency graph (e.g., package installation).	DFS naturally tracks the current path, making it easy to spot back-edges.

2.4 Time & Space Complexity — BFS and DFS

Algorithm	Time Complexity	Space Complexity	Notes
BFS	$O(V + E)$	$O(V)$	V = vertices, E = edges. Each vertex and edge is processed once.
DFS	$O(V + E)$	$O(V)$	Same as BFS. Recursion stack can be at most $O(V)$ deep.

Observations

1. The BST inorder traversal always produces a sorted sequence — this is a direct consequence of the $\text{left} < \text{root} < \text{right}$ invariant maintained at every node.
2. Deletion with two children is the most complex case: we must locate the in-order successor, copy its value, and then delete the successor (which is guaranteed to have at most one child), preserving the BST property throughout.
3. BFS and DFS produce different traversal orders because they use different data structures (queue vs. stack/recursion). For this specific graph, BFS visits F last because it is deepest from A in breadth terms.
4. The adjacency-list representation is more practical than an adjacency matrix for sparse graphs: it uses $O(V + E)$ space instead of $O(V^2)$ space.
5. Weighted edges are stored as (neighbour, weight) tuples — weights are printed in the adjacency list but are not used by basic BFS/DFS (which treat all edges equally). Dijkstra's algorithm would use the weights for shortest-path queries.

References

- Cormen, T. H. et al. *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- GeeksForGeeks — Binary Search Tree | Set 1 (Search and Insertion).
<https://www.geeksforgeeks.org/binary-search-tree-set-1-search-and-insertion/>
- GeeksForGeeks — Breadth First Search or BFS for a Graph.
<https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>
- Python Docs — collections.deque.
<https://docs.python.org/3/library/collections.html#collections.deque>